Parallel K-Means: an OpenMP vs CUDA Comparison under a Shared Parallel Design

Christian Joseph Fiore, University of Bologna, Master's Degree in Artificial Intelligence,
christian.fiore@studio.unibo.it

***Abstract*—K-Means clustering costs $O(NKD)$ operations per iteration, and most of this work is independent for every point, which makes it a natural workload for comparing parallel architectures. This work implements one parallel design, based on data parallelism by point with private reductions, both with OpenMP on a heterogeneous 12-core CPU and with CUDA on a laptop GPU, and characterizes the two versions on inputs of 100k to 1M points, with a fixed number of iterations so that every run performs the same work. On the CPU, strong scaling is sublinear: it is almost ideal on the four fast cores (efficiency ≥ 0.88), then settles at an efficiency of about 0.6, limited by load imbalance between fast and slow cores under static scheduling; weak scaling shows the same behaviour, with a single step when the slow cores are involved. On the GPU, the throughput grows with the input size and levels off at about $7 \cdot 10^{10}$ operations per second, with a fixed overhead of about 0.23 s. With the same design, the GPU is 9–13 times faster than the best CPU configuration, and 4.5–11 times faster including its overhead, with an advantage that grows with the input size.**

I. Introduction

THIS work studies how the same parallel design performs on two different architectures: a multi-core CPU, programmed with OpenMP [3], and a GPU, programmed with CUDA [4]. The workload is K-Means clustering [1], [2], starting from the serial implementation provided with the course [8]. K-Means is a good test case for this study: every iteration costs $O(NKD)$ for N points of dimension D and K clusters, most of this work is independent for every point, and the rest consists of reductions that every parallel implementation must handle.

Comparing two platforms is only meaningful if the two programs are organized in the same way; otherwise the measured differences mix the effect of the architecture with that of the algorithm. For this reason, both versions implement one shared design, and the few differences imposed by the hardware are stated explicitly. The performance is then characterized with the metrics appropriate to each architecture, and the two are compared on the same inputs.

The contributions are:

- a parallel design for K-Means based on data parallelism by point, and its mapping to OpenMP and to CUDA;
- a reproducible benchmark methodology, with controlled inputs, build and run-time settings, so that every run performs exactly the same work;
- a study of the OpenMP version (strong and weak scaling, speedup, efficiency, estimated overhead), of the CUDA version (throughput and overhead), and of the speedup of the GPU over the CPU.

Section II summarizes the algorithm, Section III describes the parallel design, Section IV the experimental setup and the metrics, Section V the results, which are discussed in Section VI. Section VII concludes.

II. Background: the K-Means Algorithm

Given N points in $\mathbb{R}^D$, K-Means looks for K centroids that minimize the sum of the squared distances between each point and its nearest centroid [2]. Lloyd's algorithm [1], used here, starts from K points chosen at random as centroids and repeats two steps: each point is assigned to the nearest centroid (squared Euclidean distance), then each centroid is replaced by the mean of the points assigned to it. It stops when no centroid moves by more than a tolerance, or after a maximum number of iterations.

In the provided implementation [8] the points are stored row by row in an $N \times D$ array of single-precision floats. The assignment step computes NK distances of D dimensions each, $O(NKD)$ operations; the update step costs $O(ND)$ to sum the points and $O(KD)$ to divide the sums and compute the shifts. The assignment step therefore dominates the cost of every iteration, and it is the main target of the parallelization.

III. Parallelization Strategy

Both implementations follow the same design. It is first described in platform-independent terms, then mapped to OpenMP and to CUDA.

A. Shared Design: Data Parallelism by Point

Every iteration of the serial code has three phases:

- *classify*: assign each point to its nearest centroid and count the points of each cluster, $O(NKD)$;
- *sum*: add up the coordinates of the points of each cluster, $O(ND)$;
- *finalize*: divide each sum by the size of its cluster, compute the shift of every centroid and the maximum shift, $O(KD)$.

Classify and *sum* are parallelized by distributing the points among the workers. In *classify*, each worker computes the distances between its points and all the K centroids and writes the cluster of each of its points. Every point is written by exactly one worker, and the points and the centroids are only read, so this phase needs no synchronization.

The two quantities that all workers update, the cluster sizes and the per-cluster sums, follow the same reduction scheme: each worker accumulates into a private copy, a barrier waits for

all the workers, and the copies are then combined. This avoids any fine-grained synchronization on shared data: the only synchronization points are the barriers between the phases.

Finalize is kept serial on purpose. It is negligible compared with *classify* ($O(KD)$ against $O(NKD)$), and it offers at most K-way parallelism: with more workers than clusters some of them would stay idle, and the cost of a parallel region with a max-reduction would exceed the gain.

With this design, the two implementations perform the same work, with the same decomposition, the same reduction scheme and the same serial fraction.

B. OpenMP Implementation

The two parallel phases are `parallel for` loops over the points with `schedule(static)`, which gives each of the T threads one contiguous block of about N/T points. Static scheduling suits the problem because every point costs the same (K distances of D dimensions); contiguous blocks also give each thread a contiguous region of memory, which is good for cache locality and avoids false sharing on the array of assignments, except at the block boundaries.

The private copies are obtained with OpenMP array reductions (`reduction(+:counts[:K])` in *classify*, `reduction(+:new_centroids[:K*D])` in *sum*): each thread accumulates into its own copy, initialized to zero, and the runtime combines the copies after the implicit barrier at the end of the loop. *Finalize* is a plain serial loop, executed outside any parallel region.

Threads are pinned to physical cores (`OMP_PROC_BIND=close`, `OMP_PLACES=cores`). Without pinning, the operating system can migrate the threads between cores, and the times of identical runs are not stable.

C. CUDA Implementation

The CUDA version uses one thread per point, in blocks of 256 threads ($\lceil N/256 \rceil$ blocks). Each iteration launches four kernels:

- `classify_kernel`: each thread finds the nearest centroid of its point. The cluster sizes are counted in a per-block histogram in shared memory, then merged into the global counts with one `atomicAdd` per cluster and block.
- `update_centroids_kernel`: each block accumulates the coordinates of its points into a private copy of the per-cluster sums in shared memory, and writes it to its own slice of a global buffer. The end of the kernel acts as the global barrier.
- `reduce_partials_kernel`: one thread per element of the sums adds that element over all the slices, in a fixed order. Every element is written by one thread, so no atomics are needed at this step; the per-block sums, built with atomics, can still differ in the last digits from run to run.
- `finalize_centroids_kernel`: the serial final phase, launched with a single thread. Running it on the GPU keeps the centroids on the device for the whole loop: at every iteration only the maximum shift (4 bytes) is copied back to the host, for the termination test.

The mapping to the shared design is direct: a CUDA thread plays the role of a worker, the end of a kernel plays the role of the barrier, and the private copies are kept per block. Three differences are imposed by the hardware:

1) *Granularity*: each thread handles one point, instead of a block of N/T points; the GPU needs many more threads than cores to hide its memory latency.
2) *Private copies per block*: a copy of the $K \times D$ sums per thread would not fit in shared memory, so the threads of a block share one copy and update it with shared-memory `atomicAdd`.
3) *Data transfers*: the input and the initial centroids must be copied to the device memory, and the results back, which has no counterpart in the shared-memory CPU version.

IV. Experimental Setup

A. Objectives and Metrics

The experiments answer three questions: (Q1) how well the OpenMP version scales with the number of threads and with the input size; (Q2) what throughput the CUDA version reaches, and how much its overhead weighs; (Q3) how much faster the GPU is than the CPU, with the same design.

All times are wall-clock times of the main loop only, without input and output, measured by the programs themselves.

For OpenMP, with $T(p)$ the time with p threads and $T(1)$, the OpenMP version run with one thread, taken as the serial time:

$$S(p) = \frac{T(1)}{T(p)}, \quad E(p) = \frac{T(1)}{pT(p)}, \quad O(p) = T(p) - \frac{T(1)}{p}. \tag{1}$$

$S(p)$ is the speedup, $E(p)$ the efficiency, and $O(p)$ the estimated overhead, i.e. the time spent above the ideal time $T(1)/p$; it includes every source of inefficiency (synchronization, load imbalance, contention for shared resources). The speedup is *linear* if $S(p) = p$ ($E = 1$) and *sublinear* if $S(p) < p$. *Strong scaling* keeps the input fixed and increases p; *weak scaling* increases the input together with p, keeping the number of points per thread N/p constant, and ideally keeps the time constant: the weak efficiency is $E_w(p) = T(N_0, 1)/T(pN_0, p)$.

For CUDA, where the number of processors cannot be chosen, the metric is the throughput:

$$\Theta = \frac{N_{\text{op}}}{T}, \qquad N_{\text{op}} = I \cdot N \cdot K \cdot D, \tag{2}$$

where I is the number of iterations and NKD the dominant asymptotic cost of one iteration (the distance terms computed in *classify*); the lower-order $O(ND)$ and $O(KD)$ terms are neglected. The programs also report an *overhead time*, measured as a whole: everything the GPU needs besides the main loop (context creation, kernel loading, device allocation, host–device copies). The throughput is reported both on T and on T plus this overhead.

TABLE I
HARDWARE PLATFORM

CPU	AMD Ryzen AI 9 HX 370 [5]
Cores	12: 4 Zen 5 + 8 Zen 5c
Clock	2 GHz base; boost 5.1 / 3.3 GHz (Zen 5 / 5c)
Threads	24 (SMT, 2 per core)
Caches	L1d 48 KB per core; L2 12 MB; L3 24 MB
Memory	32 GB LPDDR5X-7500 (2×16 GB) [6]
GPU	NVIDIA GeForce RTX 4060 Laptop [7]
SMs / cores	24 / 3072 CUDA cores
Boost clock	2370 MHz (2420 MHz OC, 140 W) [6]
GPU memory	8 GB GDDR6
Compute capability	8.9

The comparison uses the CUDA speedup $T_{\text{omp}}(p)/T_{\text{cuda}}$ for $p = 1$, 12, 24 and for the fastest OpenMP configuration p_{best}, with and without the CUDA overhead.

B. Hardware Platform

All experiments ran on an ASUS TUF Gaming A16 laptop [6] (Table I). Two features of the CPU [5] matter for the results. It is *heterogeneous*: 4 Zen 5 cores boost up to 5.1 GHz, while the other 8 are compact Zen 5c cores that boost only up to 3.3 GHz (all cores have a 2 GHz base clock). It also supports *SMT*: each core runs two hardware threads, so the 12 physical cores expose 24 logical CPUs. The GPU [7] has 3072 CUDA cores, organized in 24 streaming multiprocessors (as reported by the CUDA runtime); in this laptop it runs at a boost clock of 2370 MHz, 2420 MHz with the factory overclock, at 140 W [6].

C. Software Environment and Build

The system runs Fedora Linux 44 (kernel 7.2.7) with gcc 16.2.1 and its OpenMP runtime (libgomp), CUDA 13.4 and the NVIDIA driver 615.71.09. The programs were built without optimization or architecture flags:

```
gcc -std=c99 -Wall -Wpedantic -fopenmp \
    -DFIXED_ITERS=100 omp-k-means.c
nvcc --fmad=false -DFIXED_ITERS=100 \
    -DNO_CUDA_CHECK_ERROR cuda-k-means.cu
```

`FIXED_ITERS` makes the programs run exactly 100 iterations instead of stopping at a convergence tolerance, so that every run performs the same amount of work. `--fmad=false` prevents nvcc from fusing multiplications and additions (FMA), which gcc does not do with these flags, so both versions execute the same floating-point operations. `NO_CUDA_CHECK_ERROR` removes the device synchronization that the error-checking macro performs after every kernel launch, which has no counterpart in OpenMP.

At run time the OpenMP threads were pinned as described above, and the CUDA kernels were loaded eagerly at context creation (`CUDA_MODULE_LOADING=EAGER`), so that the first kernel launch does not pay for it inside the timed loop. No other OpenMP or CUDA environment variable was set. The laptop was plugged in, with the performance power profile. For CUDA, the runtime power management of the GPU was disabled and persistence mode enabled: otherwise the GPU is powered off while the program reads its input, and waking it up added about 2 s to the overhead of random runs.

The OpenMP program was run on every input with $p = 1, 2, 4, 6, \ldots, 24$ threads; the CUDA program was run on every input after one untimed warm-up run, which lets the driver compile and cache the GPU code. Each configuration was measured once.

D. Input Data and Reproducibility

The inputs were generated with the provided `inputgen` program [8], modified to replace the C library `rand()`, whose sequence depends on the platform, with the SplitMix64 generator [9], with its default seed 17: the same arguments produce the same file on any platform. All inputs have $D = 100$ dimensions and 10 clusters, and differ only in the number of points, $N = 100$k, 200k, 400k, 800k and 1M. The generator keeps the same cluster layout for every N, so the inputs describe the same problem at increasing size. All runs use $K = 10$ clusters, 100 iterations and centroids initialized with a fixed seed (17): every version and every thread count therefore performs exactly the same amount of work.

V. EXPERIMENTAL RESULTS

A. OpenMP Scalability

Fig. 1 shows the strong-scaling metrics for every input. The curves have the same shape for all N, with three regimes:

- *up to 4 threads*, close to the ideal: $E(2) = 0.94$–0.99, $E(4) = 0.88$–0.95;
- *at 6 threads*, a drop: the time barely changes with respect to 4 threads, within $\pm 5\%$ (e.g. $T(4) = 6.92$ s and $T(6) = 6.66$ s at 100k points), and the efficiency falls to about 0.6;
- *from 6 to 24 threads*, a roughly constant efficiency, 0.53–0.63 up to 20 threads and 0.52–0.59 at 24, so the speedup keeps growing, also beyond the 12 physical cores, up to 12.4–14.2 at 24 threads.

The estimated overhead grows in proportion to N, and its share of the running time, $1 - E(p)$, is about 37–48% from 6 threads on for every input.

Weak scaling was not measured with a dedicated experiment: it is derived from the strong-scaling measurements, which is possible because all the inputs describe the same problem at different sizes. Runs with the same number of points per thread N/p form a weak-scaling series (Table II): the 100k series groups the runs with $N/p = 100$k, from 100k points on 1 thread to 400k points on 4 threads and beyond; the 50k series groups, in the same way, the runs with $N/p = 50$k, starting from 100k points on 2 threads, so it has no single-thread run. In both series the time is almost constant up to 4 threads, increases once, by a factor of 1.5–1.6, when 8 threads are used, and is again almost constant from 8 to 20 threads.

B. CUDA Throughput

Fig. 2 shows the CUDA results. The main-loop time grows linearly with N, from 0.224 s (100k) to 1.432 s (1M). The throughput rises from $4.5 \cdot 10^{10}$ to $7.0 \cdot 10^{10}$ operations per

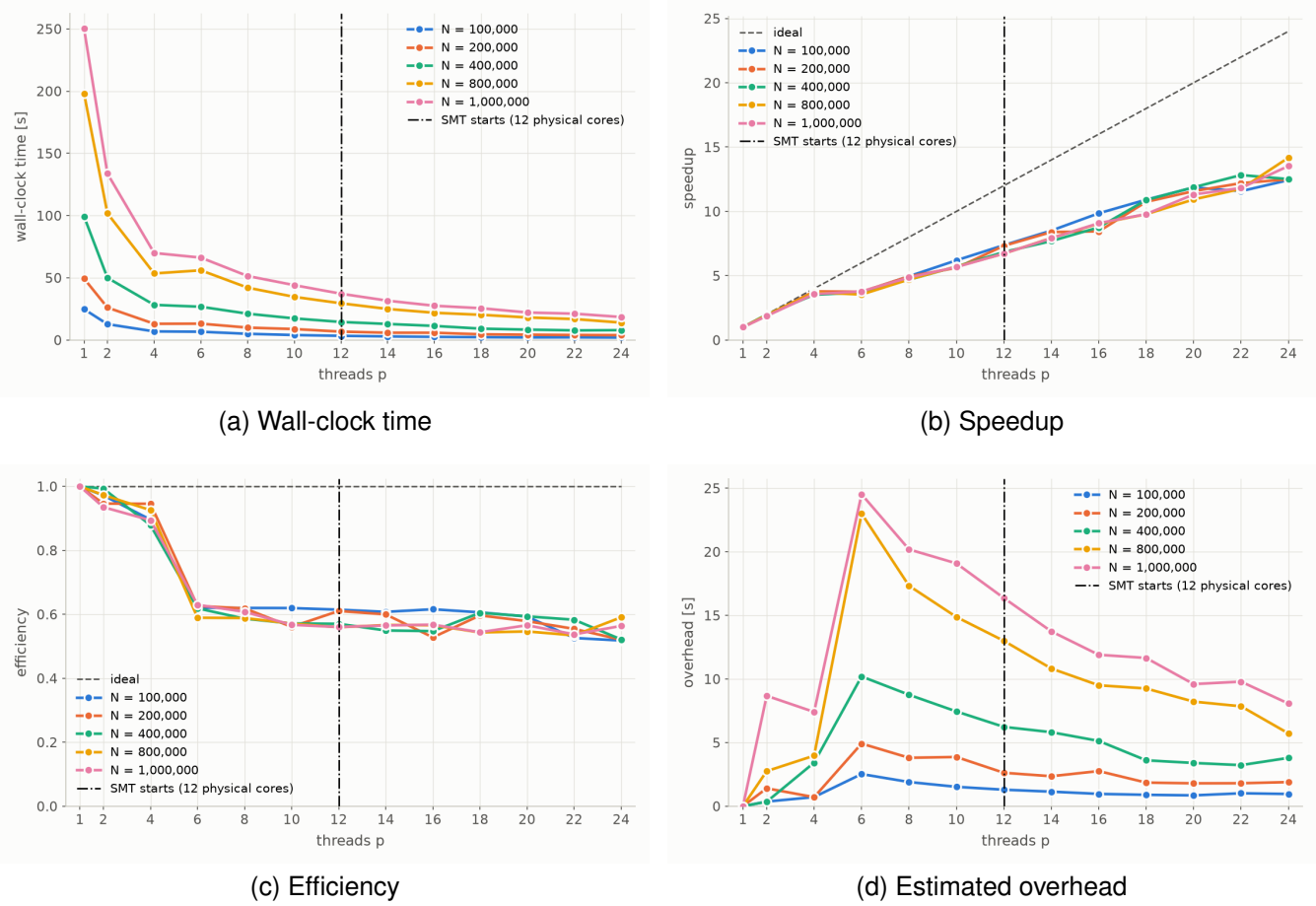


Fig. 1. OpenMP strong scaling, one line per input size N. The dash-dot line at 12 threads marks the number of physical cores: beyond it, two threads share a core (SMT).

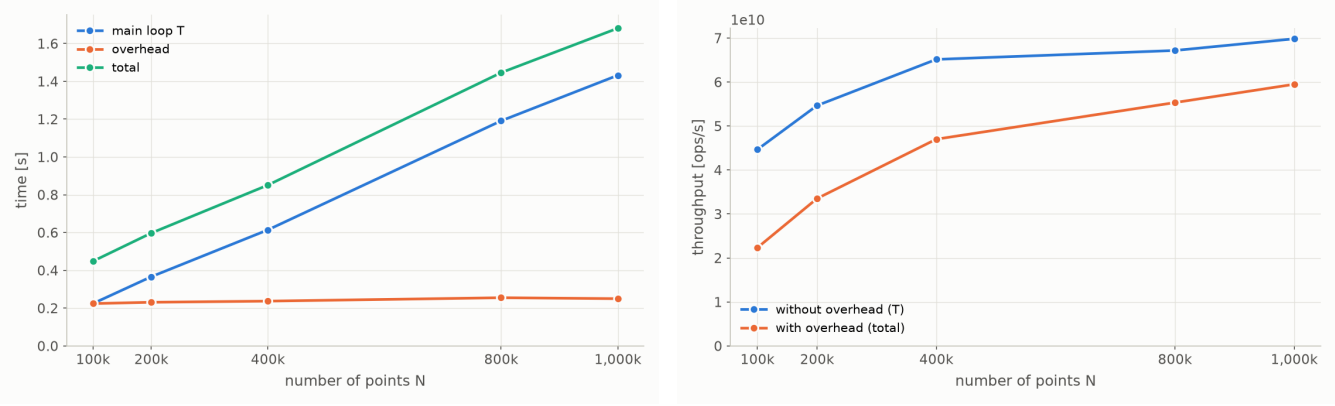
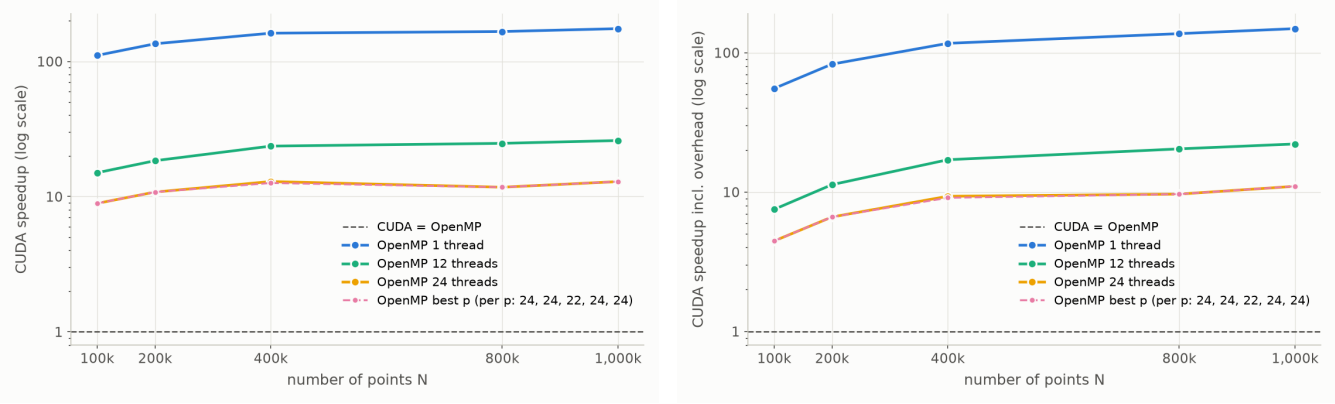


Fig. 2. CUDA time (main loop T, overhead, total) and throughput, without and with the overhead, as a function of the number of points N.

TABLE II
WEAK SCALING: TIME [S] AT CONSTANT POINTS PER THREAD, DERIVED FROM THE STRONG-SCALING RUNS

N/p	$p=1$	$p=2$	$p=4$	$p=8$	$p=10$	$p=16$	$p=20$
100k	24.8	26.1	28.2	42.1	44.1	–	–
50k	–	12.8	13.1	21.2	–	21.9	22.1

second (+56%) and levels off from 400k points on (6.5, 6.7 and $7.0 \cdot 10^{10}$). The overhead is almost constant, between 0.224 and 0.255 s; its share of the total time falls from 50% at 100k to 15% at 1M, and the throughput including the overhead rises from $2.2 \cdot 10^{10}$ to $5.9 \cdot 10^{10}$ operations per second.



(a) Main loop (b) Including the CUDA overhead

Fig. 3. CUDA speedup over OpenMP with 1, 12 and 24 threads and with the best number of threads p_{best}, as a function of N (logarithmic scale). The dashed line marks equal performance.

C. CPU vs GPU Comparison

Fig. 3 compares the two versions. The GPU is faster in every configuration: 111–175 times faster than one CPU thread, 15–26 times faster than the 12 physical cores, and 9–13 times faster than the fastest OpenMP configuration, which uses 24 threads for every input except 400k (22 threads). Including the CUDA overhead, the speedup over the best OpenMP configuration is 4.5 at 100k and 11.0 at 1M. The speedup grows with N overall: steadily against 1 and 12 threads and when the CUDA overhead is included, with a small dip at 800k against 24 threads and p_{best}.

VI. Discussion

Strong scaling is sublinear. The speedup is below p for every $p > 1$: it is almost linear only up to 4 threads ($E \geq 0.88$), and beyond 6 threads it follows $S(p) \approx 0.53$–$0.63\,p$, i.e. it still grows in proportion to p, but with a slope well below the ideal one. Amdahl's law does not explain this limit: the serial phase is negligible compared with the parallel ones, so it would still allow an almost linear speedup. The limit comes instead from the hardware (see next section).

Heterogeneous cores and static scheduling. With `OMP_PROC_BIND=close`, the first 4 threads run on the fast Zen 5 cores and the following ones on the Zen 5c cores, whose maximum clock is about 65% of the fast one (3.3 vs 5.1 GHz [5]). Static scheduling gives every thread the same number of points, so at every barrier the fast threads wait for the slow ones, and an iteration lasts as long as the slowest thread. As soon as one slow core is involved, the whole team therefore runs at the speed of the slow cores. This simple model, based on the maximum clocks, explains the two features of the curves. First, 6 threads running at the slow speed do about the same work as 4 threads at the fast speed, so $T(6) \approx T(4)$. Second, for any p that includes slow cores, every thread works at about 65% of the speed of the single fast thread that sets $T(1)$, so the efficiency is capped at about 0.65 whatever p is: this is why it stays roughly constant, slightly below this bound. The estimated overhead is thus mainly load imbalance caused by the hardware, not a cost of the OpenMP runtime: the two parallel regions per iteration typically cost microseconds, against seconds of overhead.

Weak scaling confirms it. At constant work per thread (Table II) the time is almost constant while only fast cores are used (weak efficiency $E_w = 0.95$ and 0.88 with 2 and 4 threads at 100k points per thread), which shows that the design itself scales: the synchronization cost does not grow with p or N. The single step at 8 threads, by a factor of 1.5–1.6, matches the clock ratio of the two kinds of cores (5.1/3.3 $\approx$ 1.55), and after it the time is again almost constant up to 20 threads. Weak scaling is therefore good within each kind of core, and the only loss is the step caused by the slower cores.

SMT. The speedup still grows beyond 12 threads, from 6.7–7.4 to 12.4–14.2, with almost the same efficiency. A likely explanation is that a single thread cannot keep the floating-point units of a core busy: the distance is accumulated in a chain of dependent additions, and, since the code is compiled without optimization, every operation also goes through memory. The second hardware thread would then fill these idle cycles.

CUDA throughput. The growth of the throughput with N is consistent with a fixed cost per iteration (four kernel launches, a memory reset and a small copy) that does not depend on N: it weighs less as the work per launch grows, and becomes negligible when the throughput levels off. The overhead is almost constant: from 100k to 1M points it grows by only 26 ms, while the data copied to the device grows by 360 MB, which suggests that it is dominated by a fixed cost, most likely the creation of the CUDA context. Both effects explain why the CUDA speedup grows with N.

CUDA far from the peak of the GPU. $7.0 \cdot 10^{10}$ operations per second correspond to about $2.1 \cdot 10^{11}$ floating-point operations per second (a subtraction, a multiplication and an addition per operation). From the specifications [6], [7], 3072 cores at the 2370 MHz boost clock give a peak of about 14.6 TFLOP/s counting a fused multiply-add as two operations, or 7.3 TFLOP/s without fusion, as in this build (`--fmad=false`): the kernels reach about 1.4% of the first and 2.9% of the second. Disabling the fusion lowers the

attainable peak but keeps the arithmetic identical to the CPU version. Two features of the design are the likely limits, which could be confirmed by profiling the kernels (e.g. memory efficiency and stalls on atomics).

The first is the memory access pattern. The GPU executes threads in groups of 32 (warps) that run the same instruction together, and it reads memory in segments of 32 bytes: when the 32 threads of a warp read consecutive addresses, a few segments serve the whole warp (a *coalesced* access [4]). Here each thread handles one point, and the points are stored row by row, so at each step of the distance loop consecutive threads read addresses 400 bytes apart ($D = 100$ floats). Every thread of the warp then needs its own segment, of which it uses only 4 of the 32 bytes: the kernels move much more data than they use, and only the cache partly recovers the rest of each segment in the following steps. Storing the points column by column on the device would make these reads coalesced.

The second is the contention on the shared-memory atomics that accumulate the per-cluster sums. Atomic additions to the same address cannot proceed in parallel and are executed one after the other. Each block adds the coordinates of its 256 points into only $K = 10$ rows of 100 sums, so many threads update the same addresses at the same time, and their additions are serialized. Summing within the warp before the atomic addition would reduce this contention.

Threats to validity. Each configuration was measured once, so the curves contain some noise (e.g. 22 threads slightly faster than 24 at 400k). The platform is a laptop, whose clock frequencies depend on temperature and power limits. The CPU code is compiled without optimization, while nvcc optimizes device code by default: the absolute CPU times, and therefore the CUDA speedups, would change with optimized builds. Finally, K and D were fixed ($K = 10$, $D = 100$), and the ratio between computation and memory traffic depends on them.

VII. Conclusions and Future Work

With the same parallel design, the OpenMP version shows a sublinear strong scaling: almost ideal on the 4 fast cores ($E \geq 0.88$), then at an efficiency of about 0.53–0.63, limited by the heterogeneous cores together with static scheduling; SMT still brings the speedup to 12–14 with 24 threads. Weak scaling is good within each kind of core, with a single step of 1.5–1.6 when the slow cores are involved (Q1). The CUDA version reaches a throughput of $7{\cdot}10^{10}$ operations per second, levelling off from 400k points, with an almost constant overhead of about 0.23 s, whose share falls from 50% to 15% as N grows (Q2). The GPU is 9–13 times faster than the best OpenMP configuration, and 4.5–11 times faster including its overhead, with the advantage growing with the input size (Q3).

Future work includes a fair comparison of the overheads of the two versions, an improved design (a transposed data layout for coalesced GPU accesses, and a scheduling policy that accounts for heterogeneous cores), and multi-GPU scaling.

References

[1] S. P. Lloyd, "Least squares quantization in PCM," *IEEE Trans. Inf. Theory*, vol. 28, no. 2, pp. 129–137, 1982.

[2] J. B. MacQueen, "Some methods for classification and analysis of multivariate observations," in *Proc. 5th Berkeley Symp. Math. Statist. Probab.*, vol. 1. Berkeley, CA, USA: Univ. California Press, 1967, pp. 281–297.

[3] OpenMP Architecture Review Board, "OpenMP Application Programming Interface," Version 5.2, 2021. [Online]. Available: https://www.openmp.org/specifications/. Accessed: Sep. 29, 2026.

[4] NVIDIA Corporation, "CUDA C++ Programming Guide." [Online]. Available: https://docs.nvidia.com/cuda/cuda-programming-guide/index.html. Accessed: Sep. 29, 2026.

[5] AMD, "AMD Ryzen AI 9 HX 370 – Product Specifications." [Online]. Available: https://www.amd.com/en/products/processors/laptop/ryzen/ai-300-series/amd-ryzen-ai-9-hx-370.html#product-specs. Accessed: Sep. 23, 2026.

[6] ASUSTeK Computer Inc., "ASUS TUF Gaming A16 (2024) FA608 – Tech Specs," model FA608WV. [Online]. Available: https://www.asus.com/us/laptops/for-gaming/tuf-gaming/asus-tuf-gaming-a16-2024-fa608/techspec/. Accessed: Sep. 23, 2026.

[7] NVIDIA Corporation, "GeForce RTX 40 Series Laptops – Specifications," GeForce RTX 4060 Laptop GPU. [Online]. Available: https://www.nvidia.com/en-us/geforce/laptops/40-series/. Accessed: Sep. 23, 2026.

[8] M. Marzolla, "k-means.c and inputgen.c," course material, University of Bologna, 2025.

[9] G. L. Steele, D. Lea, and C. H. Flood, "Fast splittable pseudorandom number generators," in *Proc. ACM Int. Conf. Object Oriented Program. Syst. Lang. Appl. (OOPSLA '14)*, Portland, OR, USA. New York, NY, USA: ACM, 2014, pp. 453–472, doi: 10.1145/2660193.2660195.